



INFORME TÉCNICO / TALLER DE INGENIERÍA INVERSA PARA REINGENIERÍA DE SOFTWARE

Universidad La Salle Bajío
Facultad de Ingeniería y Tecnologías
Ingeniería de Software

Sistema analizado: AuthenticationSystem

Alumno: Bardo Arion Aranda

Profesor: Javier Iván Manzanares Cuadros

Parcial: Tercer Parcial

Herramienta principal: Ghidra 12.0.4

Lenguaje analizado: Java

Fecha de entrega: 15 de mayo de 2026

Alcance del presente informe

El documento reconstruye de forma cronológica la preparación del entorno, la compilación del sistema original, el análisis de bytecode, la importación y decompilación en Ghidra, la identificación del patrón Strategy, los hallazgos de seguridad y la propuesta de reingeniería implementada con clases seguras.

Documento generado a partir del análisis del código fuente, bytecode y decompilación en Ghidra.



Diagrama de flujo general del taller

Asignatura: Ingeniería de Software

Parcial: Tercer Parcial

Herramienta principal: Ghidra 12.0.4

Lenguaje analizado: Java

Proyecto analizado: AuthenticationSystem

Fecha de entrega: 15 de mayo de 2026

Repositorio de trabajo: <https://github.com/BardoArionAranada/taller-ingenieria-inversa-reingenieria>

Índice

1. Resumen
2. Objetivo del taller
3. Alcance del análisis
4. Preparación del entorno
5. Proceso seguido paso a paso
6. Arquitectura actual del sistema
7. Análisis técnico de clases y métodos
8. Patrón de diseño identificado
9. Vulnerabilidades y code smells
10. Métricas del sistema original
11. Propuesta de reingeniería
12. Mejora implementada
13. Comparativa antes y después
14. Validación contra la guía del profesor
15. Conclusiones

1. Resumen

En este taller se realizó un proceso completo de ingeniería inversa y reingeniería sobre un sistema de autenticación escrito en Java. El trabajo comenzó con la compilación del código fuente, la generación del bytecode y el análisis del archivo .class con Ghidra. A partir de ese proceso se identificaron las clases principales, el flujo de ejecución del sistema, el patrón de diseño Strategy y varias debilidades de seguridad.

El sistema original utiliza una contraseña en texto plano y un método de "cifrado" que solo desplaza cada carácter en +2, por lo que no ofrece protección real. También mezcla responsabilidades de validación, manejo de intentos y salida en consola. Como resultado del análisis se propuso una reingeniería enfocada en seguridad, y se implementó una mejora funcional basada en hashing con SHA-256 y salt por medio de las clases SecureUser, SecureValidation y SecureAuthenticationDemo.

El resultado final cumple con la ruta principal del PDF del profesor: compilación, bytecode, análisis con Ghidra, identificación de patrón, hallazgos de seguridad, propuesta de mejora e implementación funcional.

2. Objetivo del taller

El objetivo general fue aplicar técnicas de ingeniería inversa sobre un sistema Java para:

16. Identificar clases, métodos y relaciones.
17. Reconstruir la arquitectura del sistema.
18. Identificar patrones de diseño presentes.
19. Detectar vulnerabilidades y problemas de mantenibilidad.
20. Proponer e implementar una mejora de reingeniería.

3. Alcance del análisis

El análisis cubrió:

21. Código fuente original en src/AuthenticationSystem.java.
22. Bytecode generado a partir de la compilación.
23. Archivos .class importados en Ghidra.
24. Métodos principales de autenticación y reporte.
25. Propuesta e implementación de una mejora segura.

No se consideró persistencia en base de datos, interfaz gráfica, red ni autenticación distribuida, ya que el sistema de práctica es local y de consola.

4. Preparación del entorno

Para seguir la guía del profesor se preparó una estructura de trabajo con carpetas para código, bytecode, herramientas y reportes.

Estructura final de trabajo

- 01_materiales_profesor/
- 02_herramientas/
- bin/
- ghidra_workspace/
- reports/
- src/

Herramientas utilizadas

26. JDK 21 para compilar y ejecutar el sistema.
27. VS Code para revisar código, reportes y salidas.
28. Ghidra 12.0.4 para el análisis de bytecode.
29. Git y GitHub para control de versiones del trabajo.

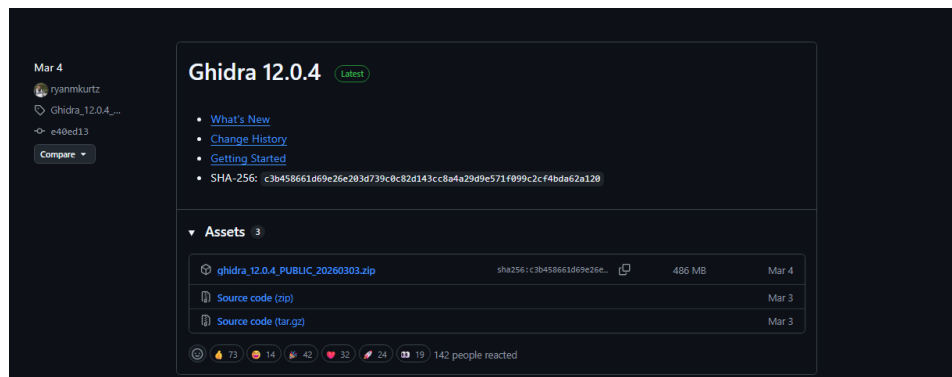


Figura 1. Versión de Ghidra 12.0.4 utilizada en el taller

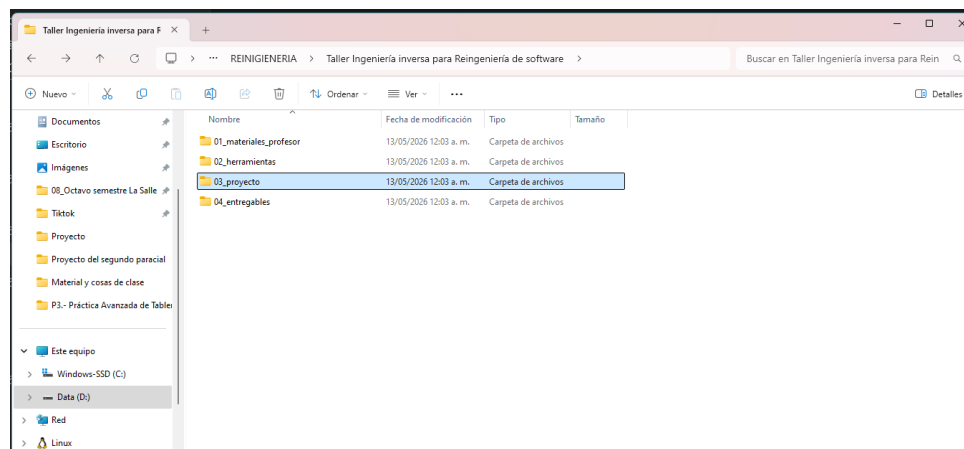


Figura 2. Estructura inicial de carpetas

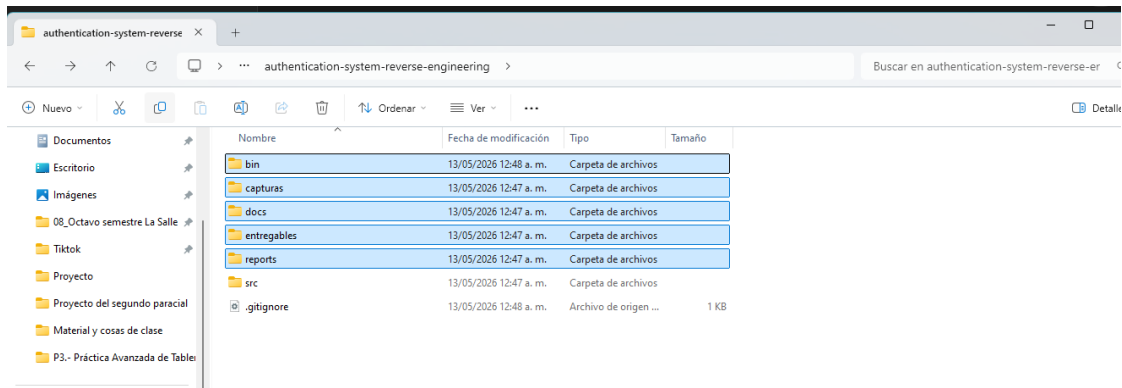


Figura 3. Estructura consolidada del repositorio

5. Proceso seguido paso a paso

5.1 Compilación y ejecución

Se compiló el sistema original con información de depuración y luego se ejecutó para comprobar su comportamiento base.

Comandos utilizados

```
javac -g src\AuthenticationSystem.java -d bin
java -cp bin AuthenticationSystem
javap -c -v bin\AuthenticationSystem.class > reports\bytecode_analysis.txt
```

La ejecución confirmó que:

- 30. El sistema crea un usuario admin.
- 31. La validación simple regresa true.
- 32. La validación fuerte regresa false.
- 33. Se genera un reporte de seguridad.

```
Windows PowerShell
Windows PowerShell
Copyright (c) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\varion> Set-Location 'D:\08_Octavo semestre La Salle\REINGENIERÍA DE SOFTWARE\REINGENIERIA\Taller Ingeniería inversa para Reingeniería de software\authentic
ation-system-reverse-engineering'
> java -version
> javac -version
> IF (!(Test-Path .\bin)) { New-Item -ItemType Directory -Name bin | Out-Null }
> javac -g src\AuthenticationSystem.java -d bin
> java -cp bin AuthenticationSystem
>
openjdk version "21.0.11" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-21.0.11+10 (build 21.0.11+10-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.11+10 (build 21.0.11+10-LTS, mixed mode, sharing)
javac 21.0.11
=== Sistema de Autenticación ===
? Validación exitosa para: admin
? Fallo de validación. Intentos: 1
Resultado Simple: true
Resultado Fuerte: false

=== REPORTE DE SEGURIDAD ===
Usuario: admin
Intentos fallidos: 1
Password (cifrado): uetgv345
Longitud password: 9
=====
PS D:\08_Octavo semestre La Salle\REINGENIERÍA DE SOFTWARE\REINGENIERIA\Taller Ingeniería inversa para Reingeniería de software\authentication-system-reverse-engineeri
ng>
```

Figura 4. Compilación y ejecución en PowerShell

5.2 Generación del bytecode

El archivo reports/bytecode_analysis.txt contiene la salida de javap -c -v. Este archivo se usó para contrastar el flujo original con la decompilación de Ghidra.

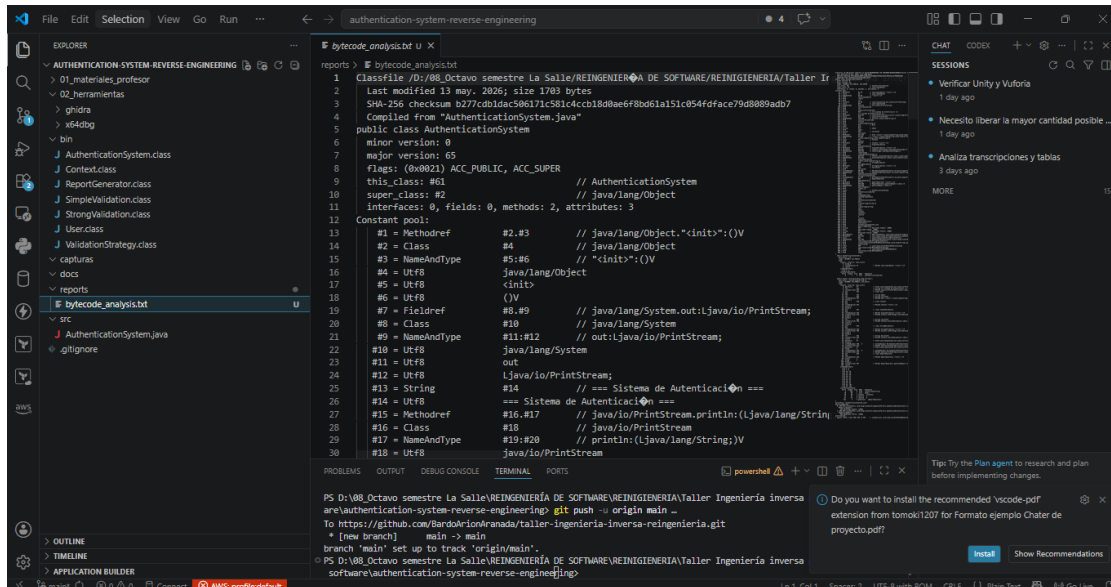


Figura 5. Bytecode en VS Code

5.3 Preparación de Ghidra

Durante la preparación se presentaron errores de Windows relacionados con la longitud de la ruta al extraer y ejecutar componentes internos de Ghidra. Esto afectó especialmente al Decompiler.

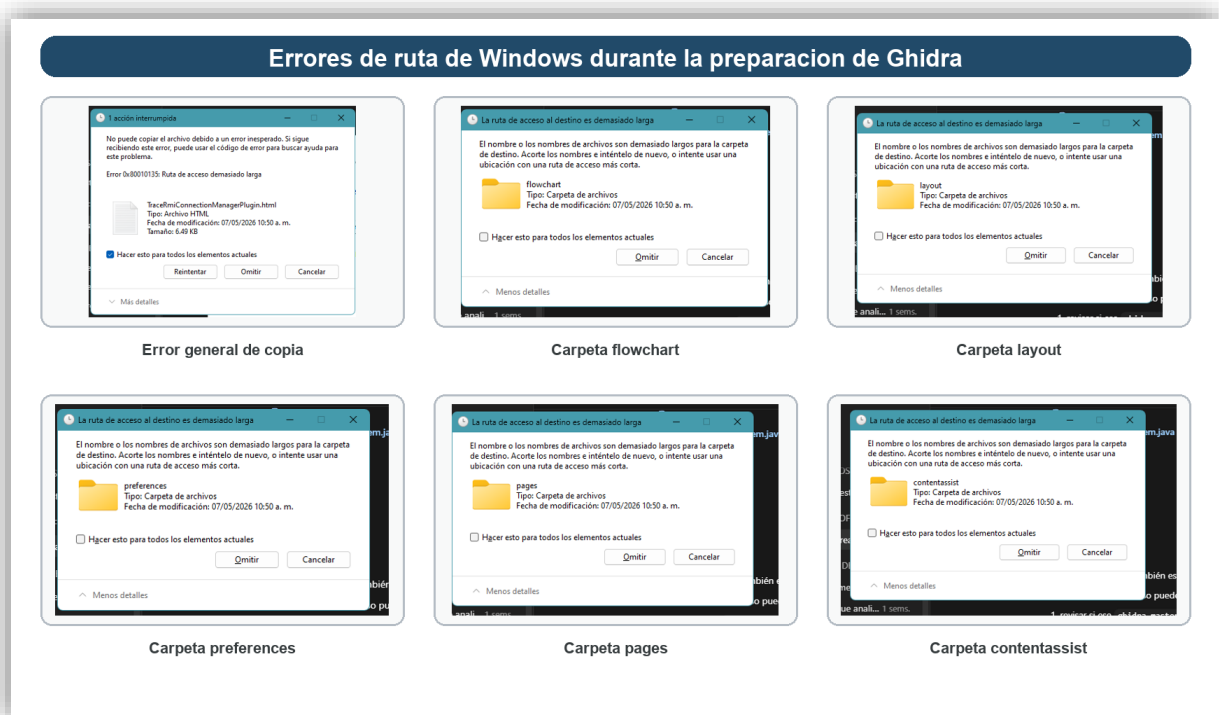


Figura 6. Error de ruta demasiado larga al copiar o extraer Ghidra

5.4 Creación del proyecto en Ghidra

Se creó un proyecto Non-Shared Project llamado Taller_Reingenieria dentro de ghidra_workspace/.



Figura 7. Pantalla principal de Ghidra

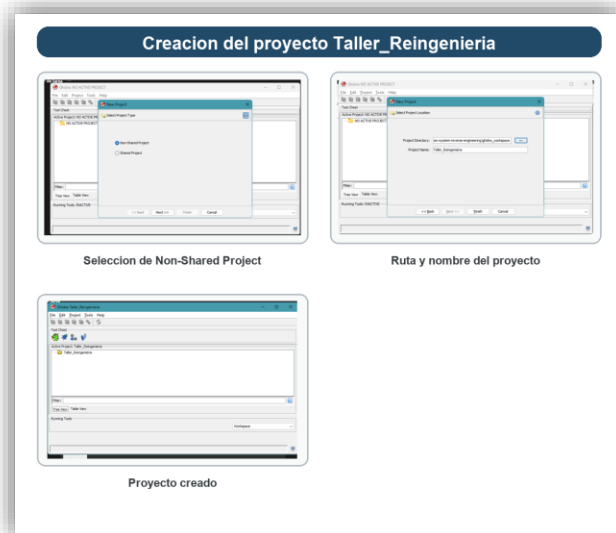


Figura 8. Creación del proyecto

5.5 Importación del archivo incorrecto

En un primer intento se seleccionó AuthenticationSystem.java en lugar de AuthenticationSystem.class. Ghidra lo detectó como Raw Binary, lo que confirmó que el archivo correcto para el taller era el compilado.

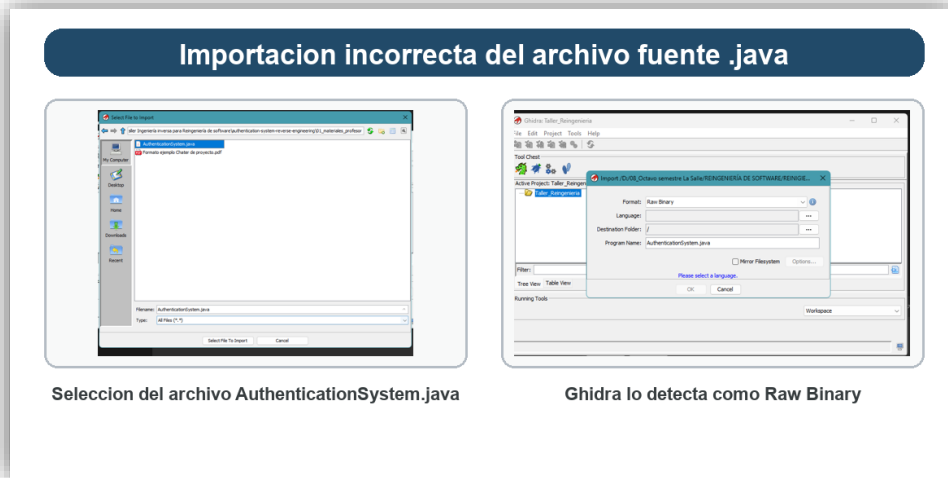


Figura 9. Importación incorrecta del archivo `.java`

5.6 Importación correcta del bytecode

Posteriormente se importó correctamente bin/AuthenticationSystem.class con formato Java Class File, lenguaje JVM:BE:32:default:default y análisis automático activado.

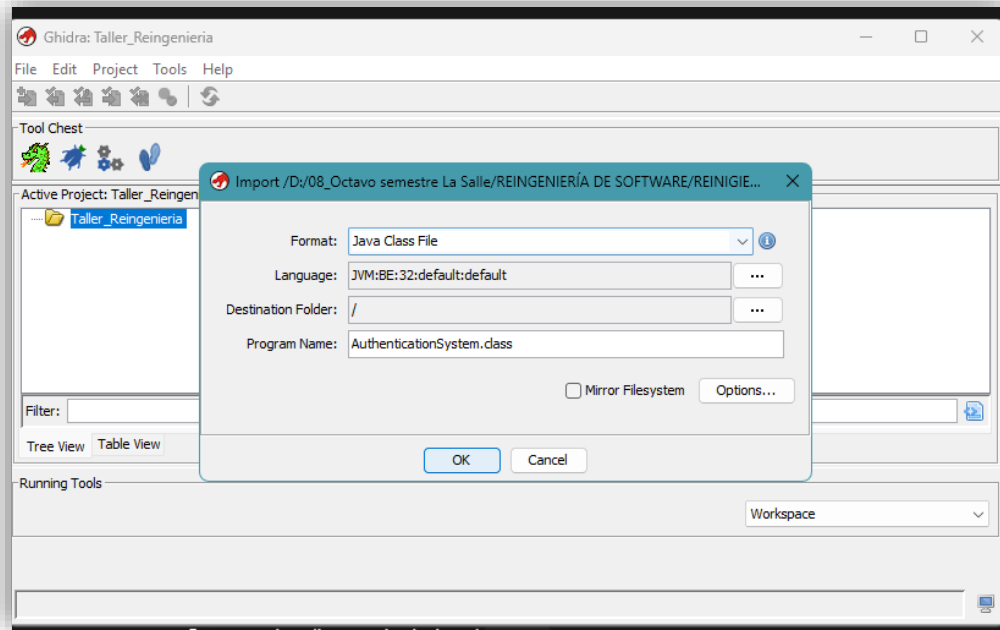


Figura 10. Importación correcta del `.class`

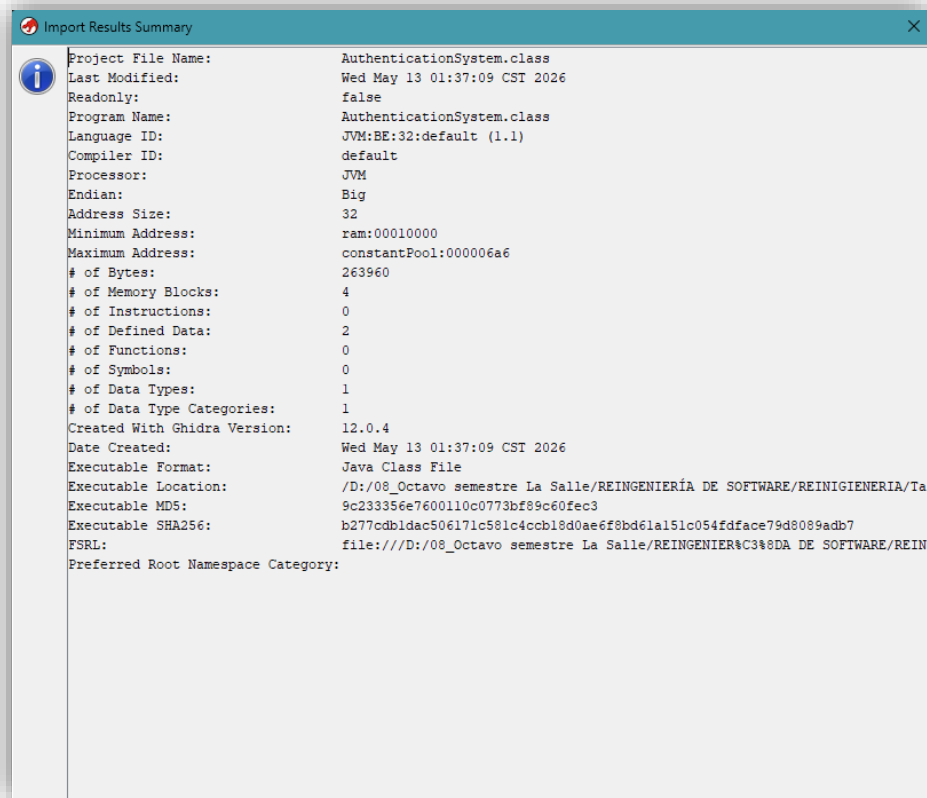


Figura 11. Import Results Summary

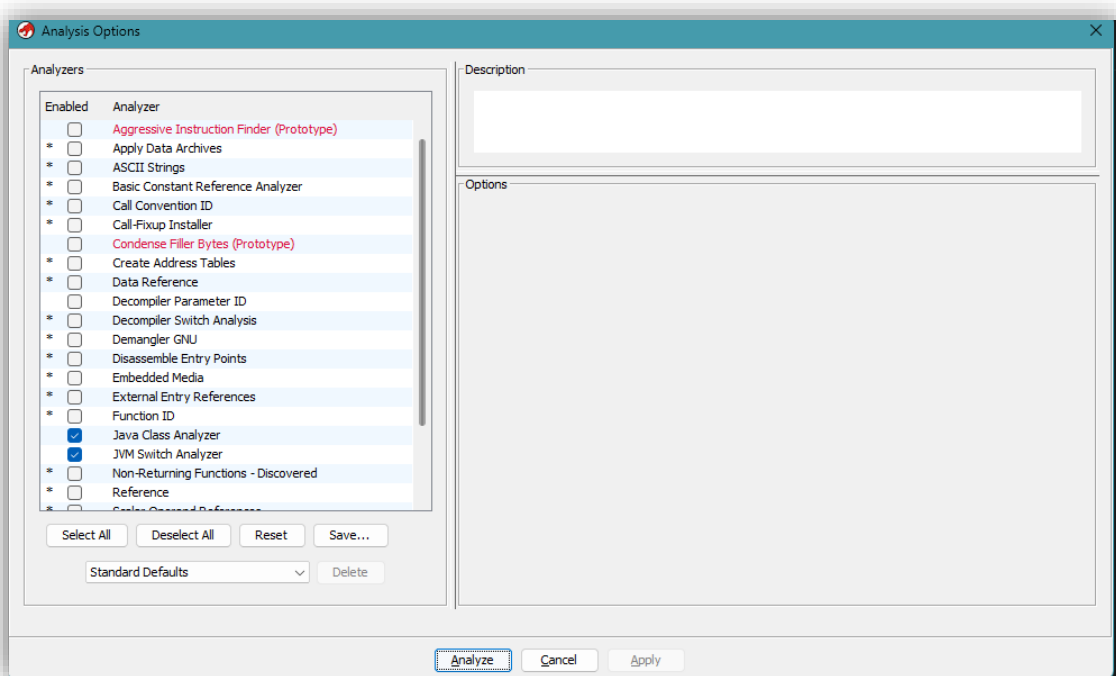


Figura 12. Analysis Options

5.7 Importación del resto de clases

Después se importaron:

- 34. Context.class
- 35. ReportGenerator.class
- 36. SimpleValidation.class
- 37. StrongValidation.class
- 38. User.class
- 39. ValidationStrategy.class

Esto permitió reconstruir el sistema completo y navegar entre sus componentes.

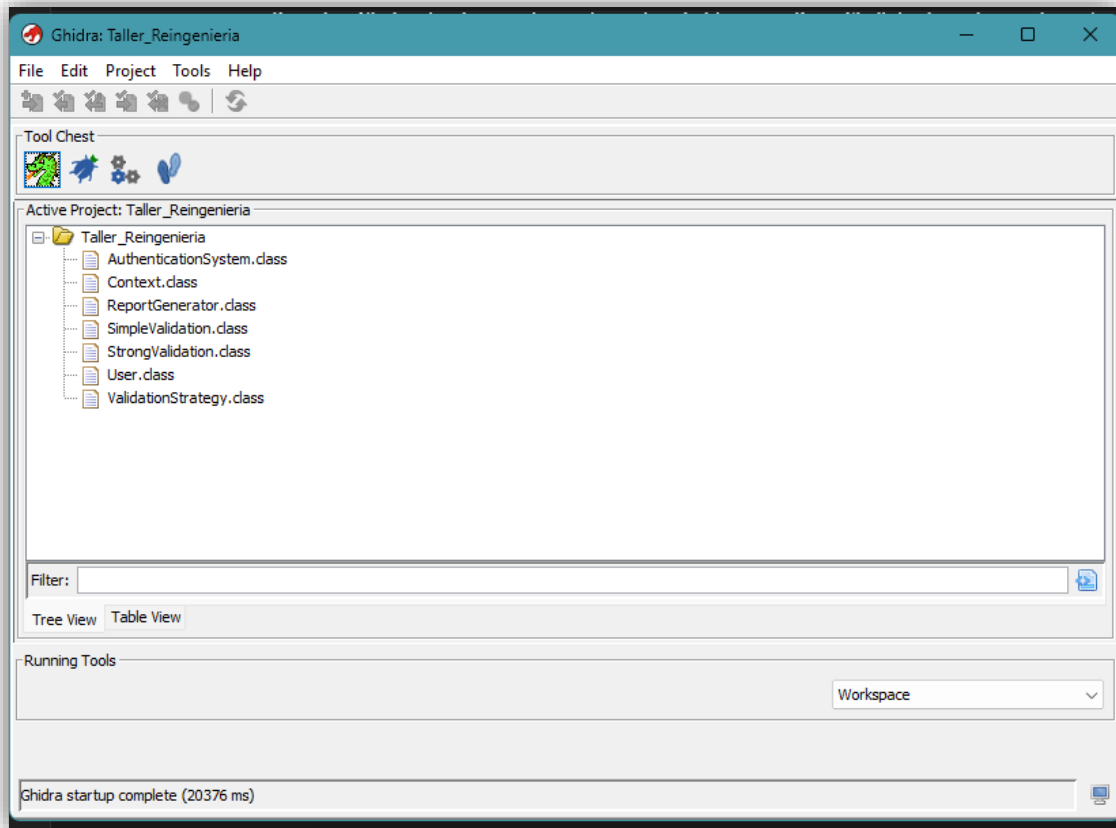


Figura 13. Árbol del proyecto con las 7 clases

5.8 Solución del problema del Decompiler

El Decompiler fallaba porque el ejecutable decompile.exe quedaba debajo de una ruta demasiado larga. La solución práctica fue abrir Ghidra desde una ruta corta temporal montada en G: para permitir que el decompilador arrancara correctamente.

Este hallazgo también es relevante para el reporte porque demuestra una incidencia técnica real del proceso.

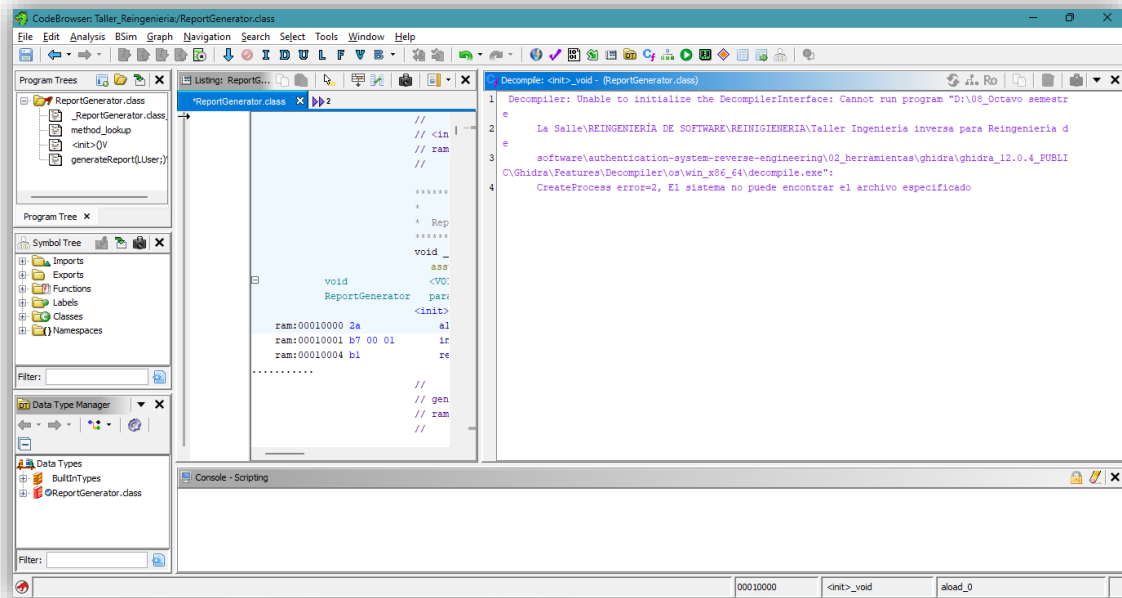


Figura 14. Error del decompiler por longitud de ruta

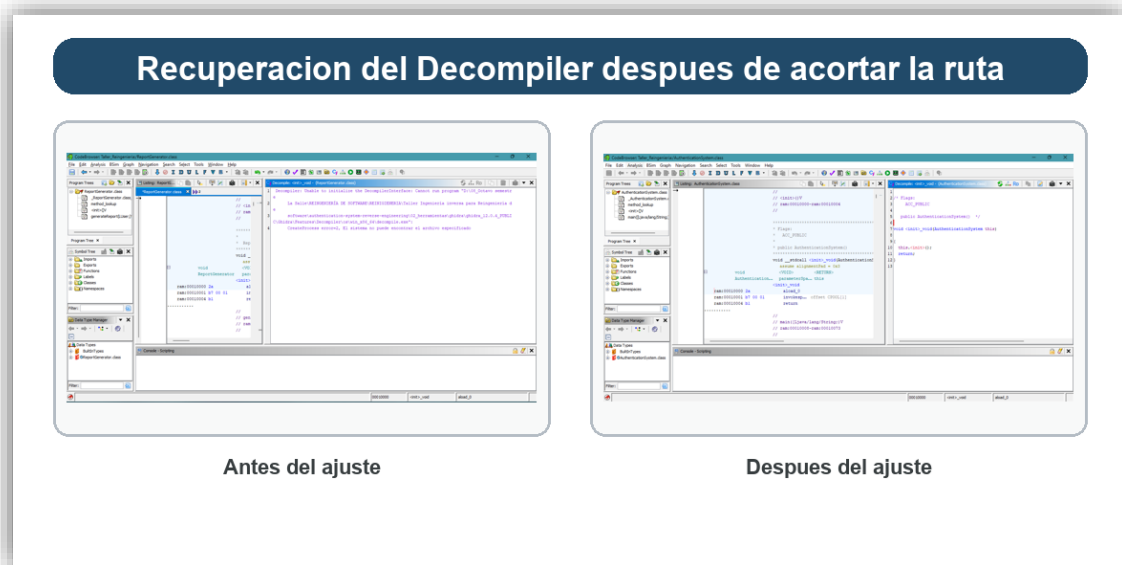


Figura 15. Decompiler funcionando después de corregir la ruta

6. Arquitectura actual del sistema

El sistema original está compuesto por 7 tipos relevantes:

- 40. AuthenticationSystem
- 41. User
- 42. ValidationStrategy
- 43. SimpleValidation
- 44. StrongValidation
- 45. Context
- 46. ReportGenerator

6.1 Diagrama de clases original

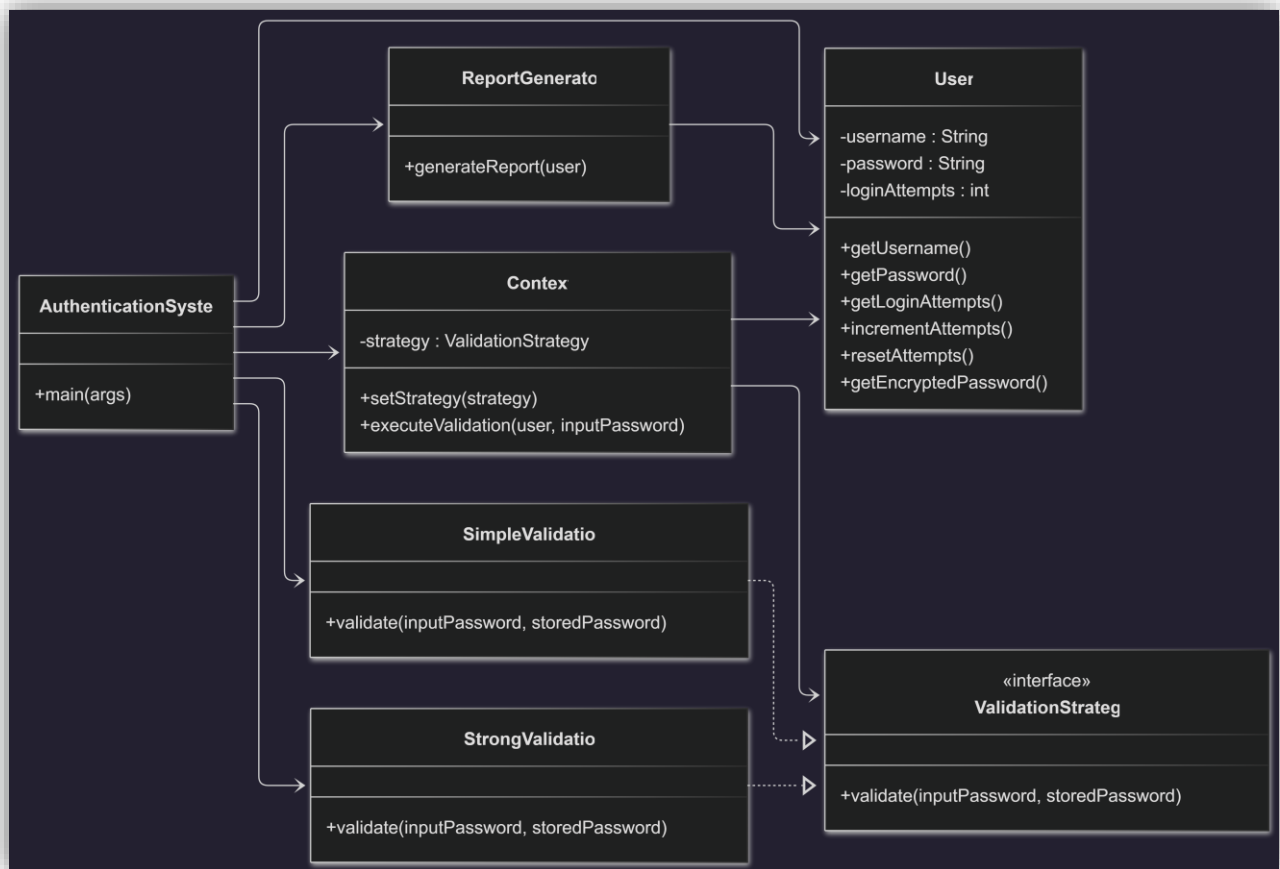


Diagrama generado para la sección 1

6.2 Relación general del sistema

La clase AuthenticationSystem coordina el flujo principal. Context delega la decisión de validación a una estrategia concreta. User encapsula el estado del usuario y ReportGenerator imprime un resumen en consola.

7. Análisis técnico de clases y métodos

7.1 `AuthenticationSystem.main()`

El método main() realiza el flujo principal:

47. Imprime el título del sistema.
48. Crea un usuario con credenciales fijas.
49. Crea un contexto de validación.
50. Ejecuta SimpleValidation.
51. Ejecuta StrongValidation.
52. Muestra ambos resultados.
53. Genera un reporte final.

Conclusiones:

54. El flujo general del sistema es simple y lineal.
55. El sistema está preparado para intercambiar reglas de validación.
56. El método depende directamente de varias clases concretas.

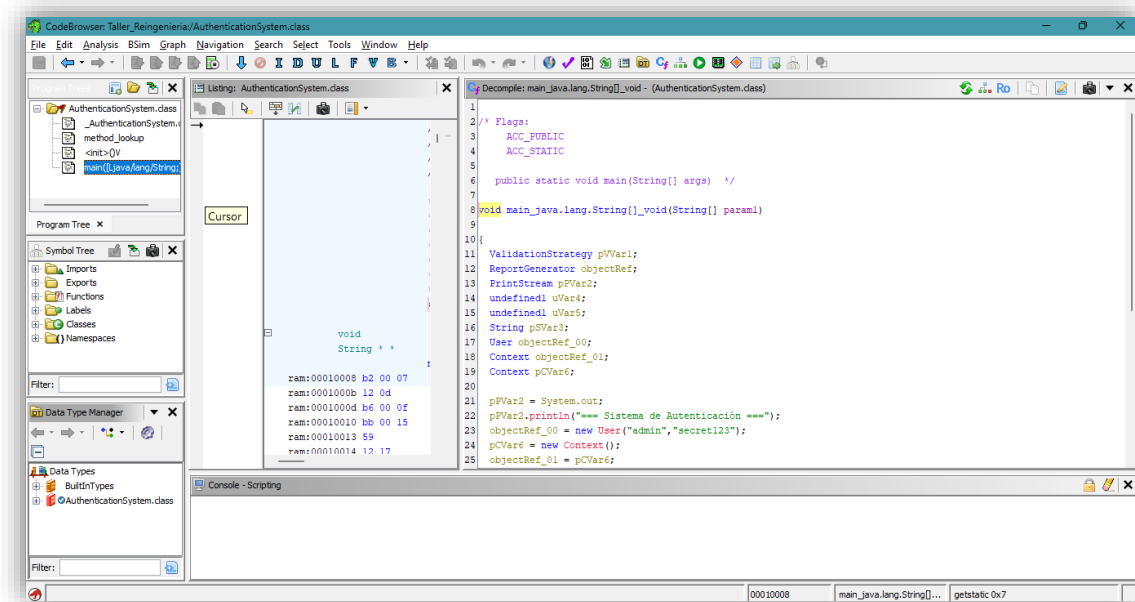


Figura 16. Decompilación de `main()`

7.2 `User.getEncryptedPassword()`

Este método convierte la contraseña a un arreglo de caracteres y suma +2 a cada carácter antes de construir una nueva cadena.

```
for (char c : password.toCharArray()) {  
    encrypted.append((char) (c + 2));  
}
```

Conclusiones:

- 57. No existe cifrado real.
- 58. La transformación es reversible.
- 59. Equivale a un cifrado César con desplazamiento +2.

Esto representa una vulnerabilidad fuerte porque cualquier atacante que vea el algoritmo puede descifrarlo de inmediato restando 2 a cada carácter.

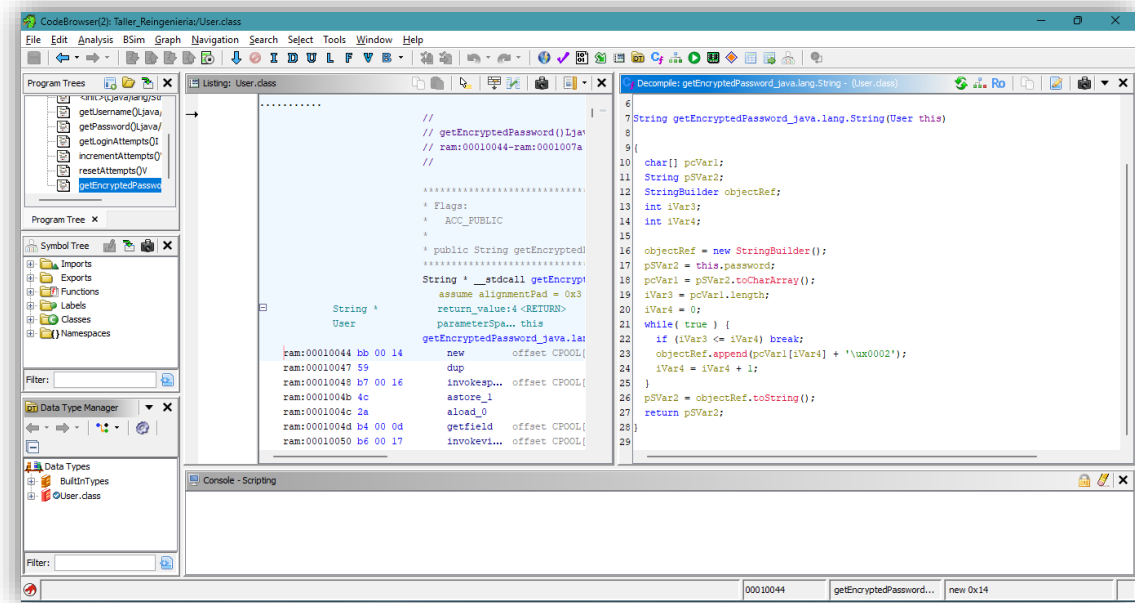


Figura 17. Decompilación de `getEncryptedPassword()`

7.3 `Context.executeValidation()`

El método `executeValidation()` concentra la lógica de autenticación:

- 60. Verifica que exista una estrategia configurada.
- 61. Llama a `strategy.validate(inputPassword, user.getPassword())`.
- 62. Si la validación es correcta, reinicia los intentos.
- 63. Si la validación falla, incrementa el contador.
- 64. Imprime mensajes de éxito o fallo.

Conclusiones:

- 65. Context representa claramente el punto de uso del patrón Strategy.
- 66. La clase delega la validación pero mezcla más de una responsabilidad.
- 67. Esta clase controla autenticación, estado e impresión por consola en el mismo método.

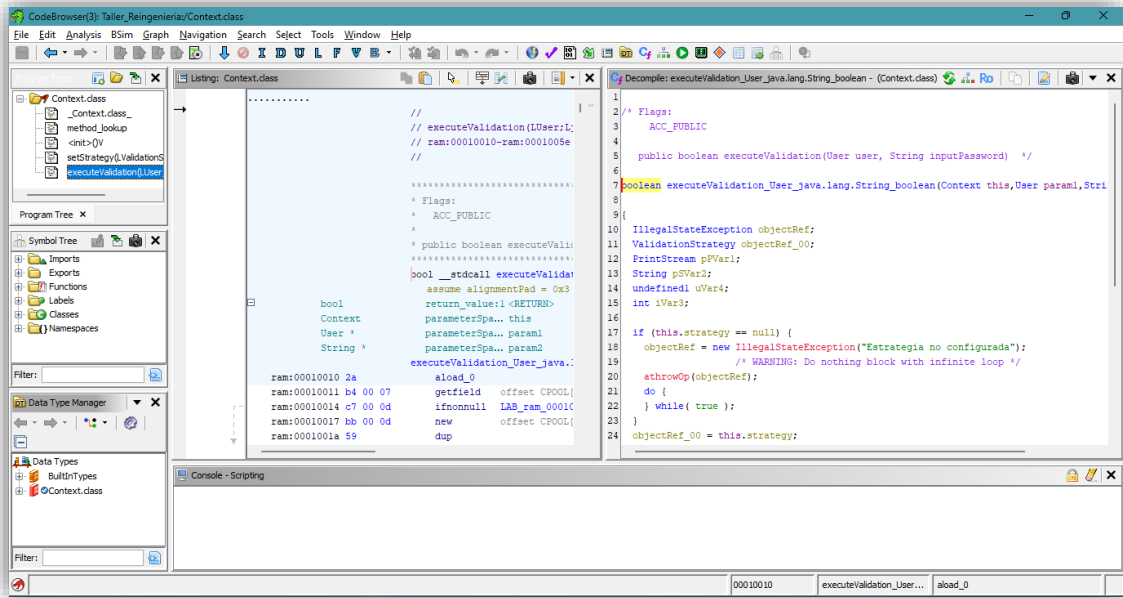


Figura 18. Decompilación de `executeValidation()`

7.4 `ReportGenerator.generateReport()`

El método generateReport() imprime:

- 68. Usuario.
- 69. Intentos fallidos.
- 70. Password "cifrado".
- 71. Longitud de la contraseña.

Conclusiones:

- 72. El reporte expone información sensible.
- 73. Mostrar la contraseña transformada sigue siendo una fuga de datos.
- 74. Mostrar la longitud de la contraseña también entrega información innecesaria.

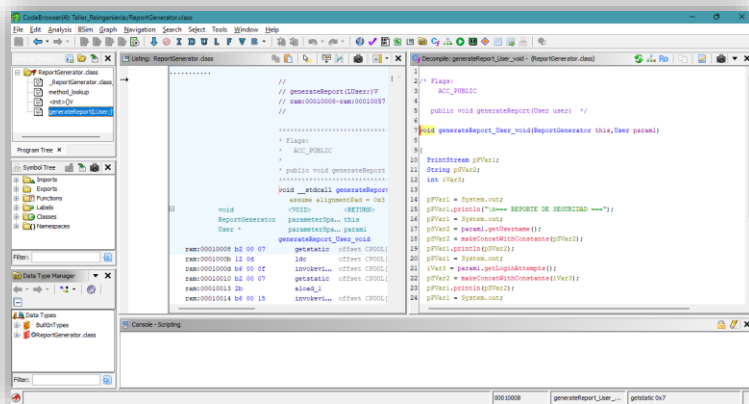


Figura 19. Decompilación de `generateReport()`

7.5 `SimpleValidation.validate()`

La estrategia simple solo verifica dos condiciones:

- 75. Que ambas cadenas no sean null.
- 76. Que ambas cadenas sean iguales con equals.

Conclusiones:

- 77. Es la estrategia más básica posible.
- 78. No valida fortaleza, longitud ni estructura.
- 79. Es funcional, pero insegura para un sistema real.

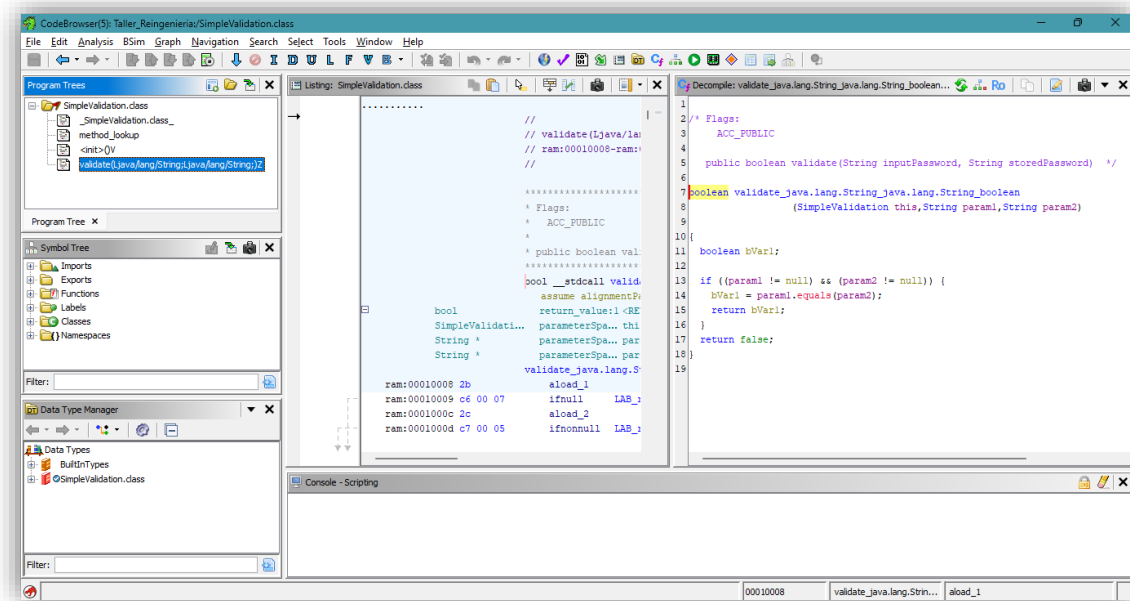


Figura 20. Decompilación de `SimpleValidation.validate()`

7.6 `StrongValidation.validate()`

La estrategia fuerte agrega reglas de complejidad:

- 80. Rechaza null.
- 81. Exige longitud mínima de 8.
- 82. Exige mayúscula, minúscula y número.
- 83. Finalmente compara con equals.

Conclusiones:

- 84. Es más estricta que la validación simple.
- 85. Aún depende de que la contraseña almacenada esté en texto plano.
- 86. Mejora la calidad de la entrada, pero no resuelve el problema de almacenamiento inseguro.

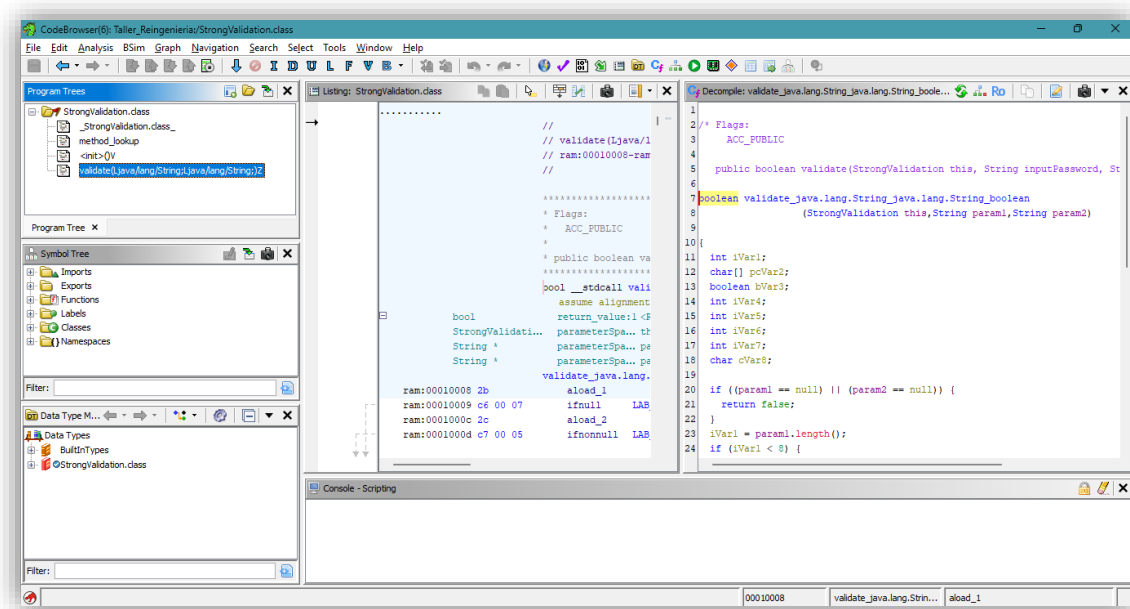


Figura 21. Decompilación de `StrongValidation.validate()`

8. Patrón de diseño identificado

El patrón identificado es Strategy.

8.1 Evidencia

87. Existe una interfaz ValidationStrategy.

88. Existen dos implementaciones concretas:

SimpleValidation y StrongValidation.

89. La clase Context mantiene una referencia a una estrategia.

90. La estrategia puede cambiarse en tiempo de ejecución con setStrategy(...).

8.2 Interpretación

El sistema fue diseñado para cambiar la regla de validación sin modificar la clase Context. Esto es correcto desde el punto de vista de flexibilidad, pero la implementación sigue arrastrando problemas de seguridad porque la contraseña base se almacena en texto plano.

8.3 Diagrama del patrón

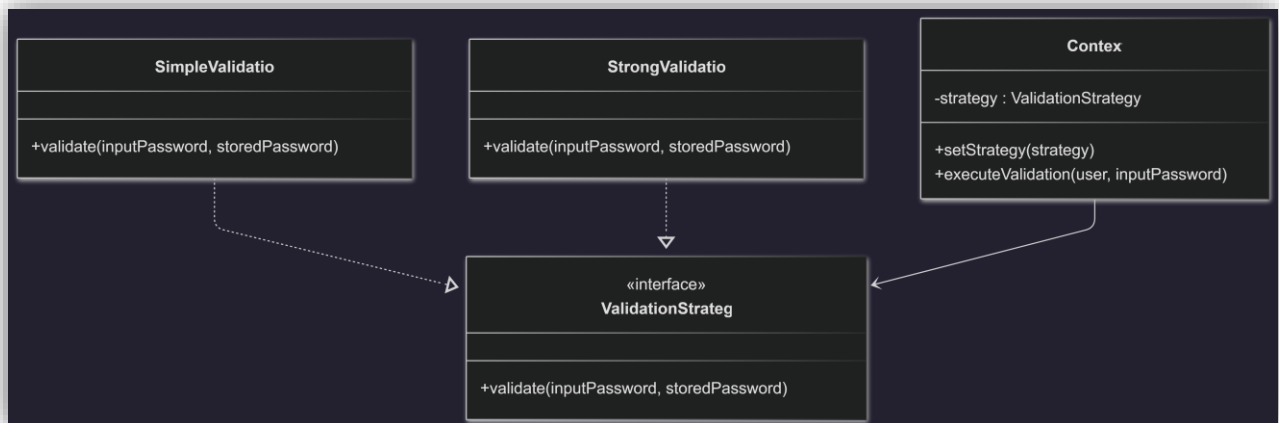


Diagrama generado para la sección 2

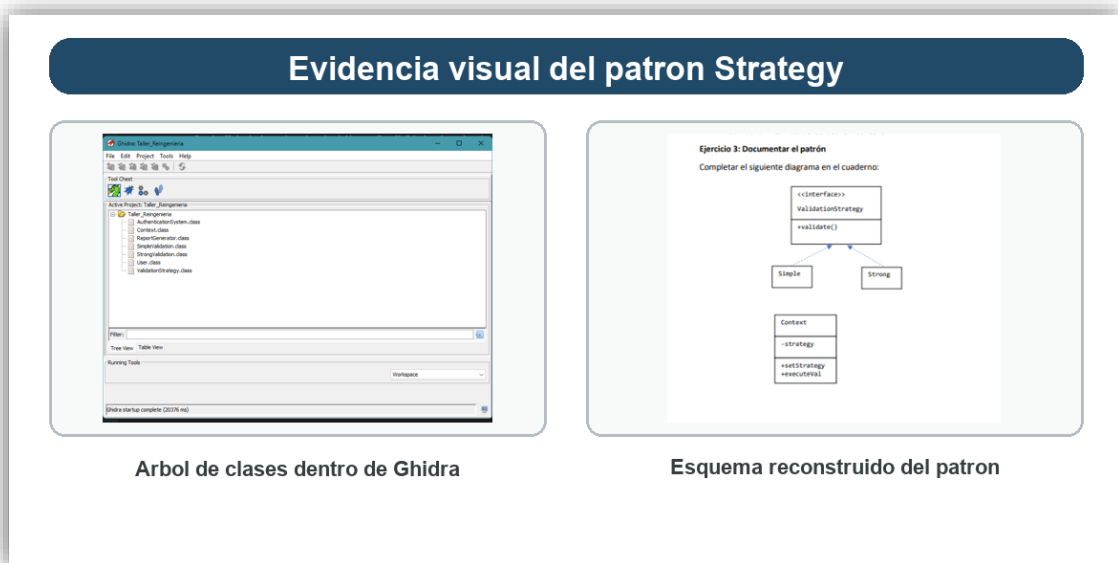


Figura 22. Evidencia visual del patrón Strategy

9. Vulnerabilidades y code smells

9.1 Vulnerabilidades encontradas

91. Contraseña en texto plano

User almacena directamente la contraseña en el atributo password.

92. Cifrado débil y reversible

getEncryptedPassword() usa un desplazamiento +2 por carácter.

93. Exposición de información sensible

ReportGenerator imprime password transformado y longitud.

94. Sin bloqueo real de cuenta

El sistema aumenta intentos fallidos pero no aplica bloqueo.

95. Sin hash ni salt

No existe almacenamiento seguro de credenciales.

9.2 Code smells detectados

96. Responsabilidad mezclada en `Context`

Valida, cambia estado del usuario e imprime mensajes.

97. Acoplamiento alto en `main()`

AuthenticationSystem conoce demasiadas clases concretas.

98. Duplicación de validaciones nulas

SimpleValidation y StrongValidation repiten la validación de null.

99. Uso de datos de prueba quemados

Las credenciales admin y secret123 están embebidas en el flujo principal.

100. Salida directa por consola

No existe una capa de logging o reportes desacoplada.

10. Métricas del sistema original

10.1 Conteo general

Métrica	Valor	Interpretación
Número de tipos principales	7	El sistema es pequeño y fácil de rastrear
Líneas totales del archivo original	151	Sistema corto, apropiado para práctica
Profundidad de herencia	0	No hay jerarquía de herencia entre clases originales
Interfaces	1	Se usa una abstracción para las validaciones
Estrategias concretas	2	Existe variación de comportamiento

10.2 Métodos por clase

Clase	Métodos definidos en fuente	Observación
AuthenticationSystem	1	Solo `main()`
User	7	Constructor, getters, mutadores y cifrado débil
ValidationStrategy	1	Método abstracto `validate(...)`
SimpleValidation	1	Validación básica
StrongValidation	1	Validación con reglas adicionales
Context	2	Cambia estrategia y ejecuta validación
ReportGenerator	1	Emite el reporte final

10.3 Acoplamiento y cohesión

Métrica	Valor estimado	Justificación
Acoplamiento	Medio	`main()` depende de varias clases concretas
Cohesión en `User`	Media	Mezcla datos de usuario con cifrado reversible
Cohesión en `Context`	Media-Baja	Mezcla estrategia, flujo y salida
Cohesión en `ReportGenerator`	Media	Solo reporta, pero expone datos inseguros

11. Propuesta de reingeniería

11.1 Objetivo de la mejora

La mejora propuesta busca eliminar el almacenamiento inseguro de contraseñas y reemplazarlo por un esquema más apropiado:

101. Generar un salt aleatorio por usuario.
102. Calcular un hash SHA-256 de password + salt.
103. Almacenar solo salt y hash.
104. Validar comparando hashes, no texto plano.

11.2 Cambios propuestos

105. Crear una clase SecureUser.
106. Crear una estrategia SecureValidation.
107. Mantener Context para aprovechar el patrón Strategy.
108. Agregar una clase demo para mostrar el comportamiento mejorado.

11.3 Diagrama propuesto

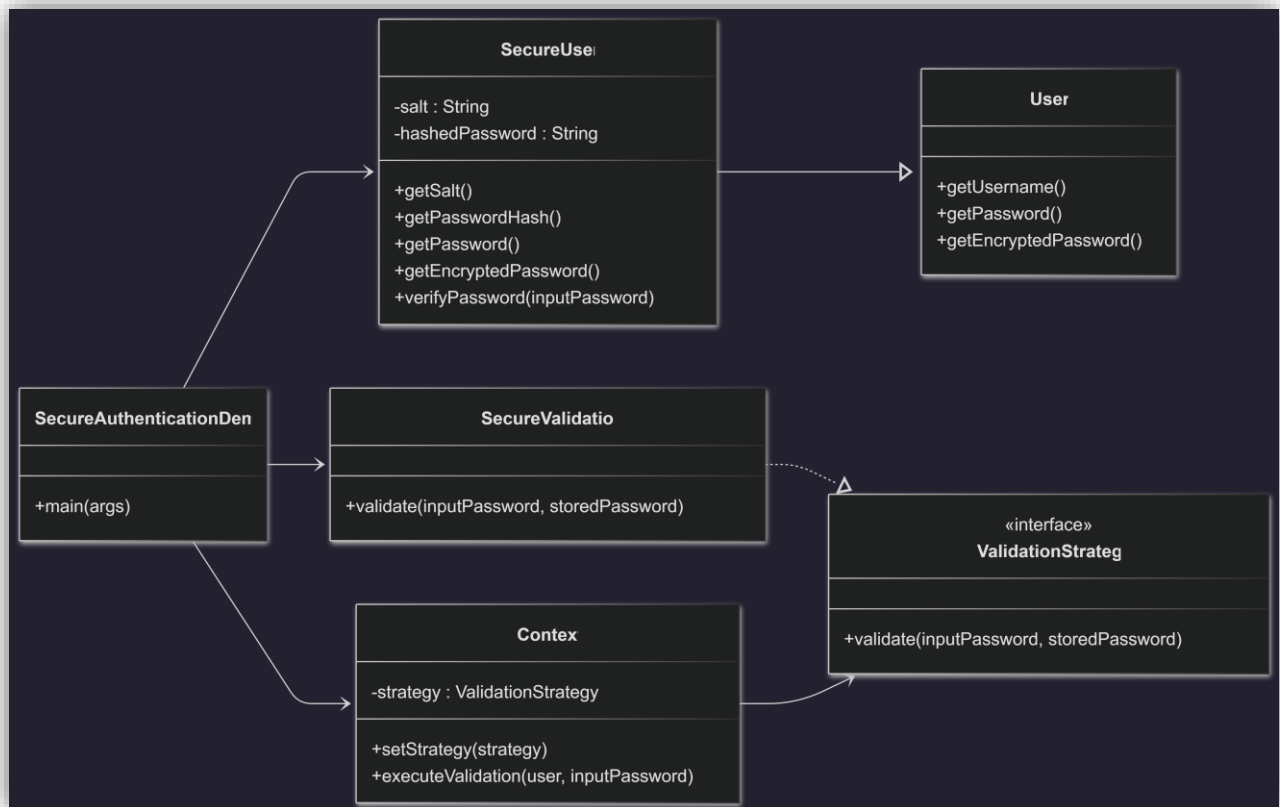


Diagrama generado para la sección 3

12. Mejora implementada

12.1 Archivos agregados

Se implementaron los siguientes archivos:

- 109. `src/SecureUser.java`
- 110. `src/SecureValidation.java`
- 111. `src/SecureAuthenticationDemo.java`

12.2 `SecureUser`

La clase **SecureUser** hereda de **User**, genera un salt aleatorio de 16 bytes y produce un hash SHA-256 codificado en Base64.

Mejoras clave:

- 112. La contraseña original no se conserva como texto plano.
- 113. El método `getPassword()` regresa salt:hash.
- 114. El método `getEncryptedPassword()` ahora devuelve el hash.
- 115. Se incorpora `verifyPassword(...)` para pruebas directas.

12.3 `SecureValidation`

La clase SecureValidation implementa ValidationStrategy y:

116. Divide el valor almacenado usando :.
117. Recupera el salt.
118. Genera el hash de entrada.
119. Compara el hash calculado contra el hash almacenado.

12.4 `SecureAuthenticationDemo`

La clase demo prueba la mejora con:

120. Una autenticación correcta.
121. Una autenticación incorrecta.
122. Un reporte final usando el mismo Context.

12.5 Validación técnica de la mejora

La mejora fue compilada y ejecutada correctamente. La salida observada fue consistente con el comportamiento esperado:

123. La contraseña correcta fue aceptada.
124. La contraseña incorrecta fue rechazada.
125. El reporte mostró un hash en lugar de la contraseña original.

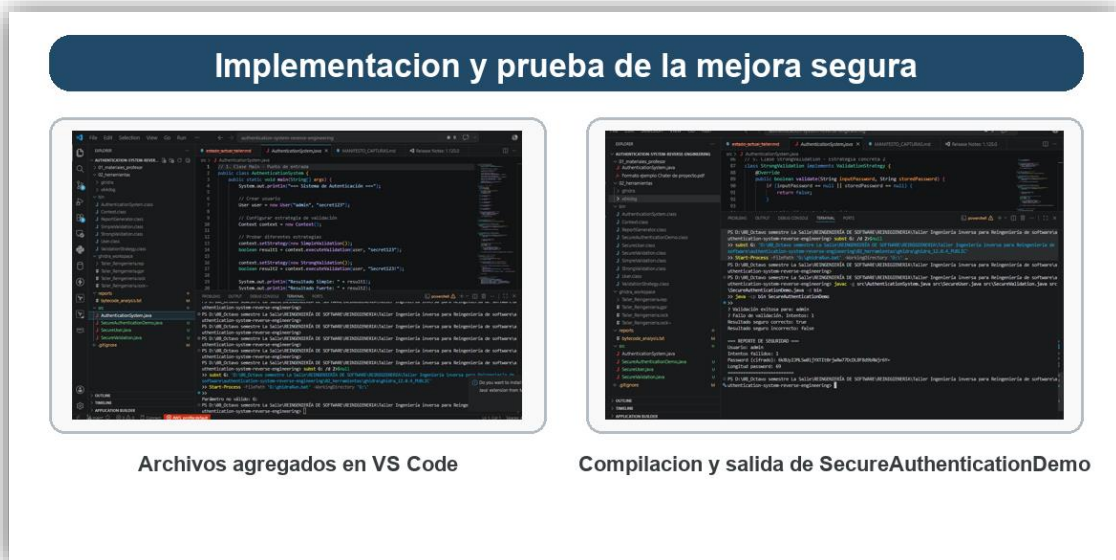


Figura 23. Ejecución de la mejora segura

13. Comparativa antes y después

Aspecto	Versión original	Versión mejorada	Resultado
Almacenamiento de contraseña	Texto plano	Hash `SHA-256` con `salt`	Mejora alta
Método de "cifrado"	Desplazamiento `+2`	Hash irreversible	Mejora alta
Validación	`equals()` y reglas básicas	Comparación de hash	Mejora alta
Exposición de datos	Muestra password transformado	Muestra hash	Mejora media
Reutilización del patrón	Strategy	Strategy conservado	Correcto
Mantenibilidad	Media	Media-Alta	Mejorada

14. Validación contra la guía del profesor

14.1 Checklist de cumplimiento

Punto de la guía	Estado	Evidencia
Estructura del proyecto	Cumplido	Carpetas de código, reportes, bytecode y workspace
Compilación con `javac -g`	Cumplido	`bin/` generado con archivos `.class`
Ejecución del sistema original	Cumplido	Salida observada en terminal
Generación de bytecode con `javap`	Cumplido	`reports/bytecode_analysis.txt`
Importación del `.class` en Ghidra	Cumplido	Importación correcta como `Java Class File`
Auto Analyze en Ghidra	Cumplido	Captura de opciones de análisis
Análisis de `main()`	Cumplido	Decompilación revisada
Análisis de `getEncryptedPassword()`	Cumplido	Decompilación revisada
Análisis de `executeValidation()`	Cumplido	Decompilación revisada
Análisis de `generateReport()`	Cumplido	Decompilación revisada
Identificación del patrón Strategy	Cumplido	Interfaz, implementaciones y contexto
Detección de vulnerabilidades	Cumplido	Se documentaron 5 hallazgos

		principales
Tabla de métricas	Cumplido	Se incluye en este informe
Propuesta de reingeniería	Cumplido	Sección 11
Implementación de mejora	Cumplido	`SecureUser`, `SecureValidation`, `SecureAuthenticationDemo`
Presentación de 5 a 7 minutos	Pendiente externo	No forma parte de este archivo; se prepara aparte

15. Conclusiones

El sistema original fue útil para demostrar un caso pequeño pero claro de ingeniería inversa. Ghidra permitió reconstruir el flujo del programa, reconocer el patrón Strategy e identificar problemas importantes de seguridad y diseño.

La vulnerabilidad más importante fue el almacenamiento de contraseñas en texto plano y el uso de un desplazamiento +2 en `getEncryptedPassword()`, que no representa un cifrado real. También se detectó que el sistema expone datos sensibles en consola y concentra demasiadas responsabilidades en Context.

La reingeniería propuesta y aplicada mejora el sistema sin destruir su arquitectura general. En lugar de reescribir todo, se mantuvo el patrón Strategy y se sustituyó la lógica insegura por un esquema con salt y SHA-256. Esto demuestra que la reingeniería puede conservar partes valiosas del sistema mientras corrige debilidades críticas.

En conclusión, el taller cumplió con el objetivo de pasar de la observación del bytecode a una mejora funcional justificada técnicamente.